

To Begin with the Lab

First Jenkins file

Summary of the Lab

This lab introduces users to creating a basic Jenkins pipeline job. A new item is created and configured as a pipeline project. Using the built-in **Pipeline Script** definition, a sample pipeline is selected and saved. Once saved, the job is triggered using the **Build Now** option. Jenkins provides visual feedback using the **Stage View**, and users can monitor each stage's progress. Finally, the **Console Output** gives detailed execution logs. This setup acts as the foundation for more complex pipeline configurations, providing hands-on experience with Jenkins' pipeline structure and job execution process.

- Create a new item.
- Give the name of the job
- Select the pipeline
- Click OK.

Dashboard > All > New Item

New Item

Enter an item name

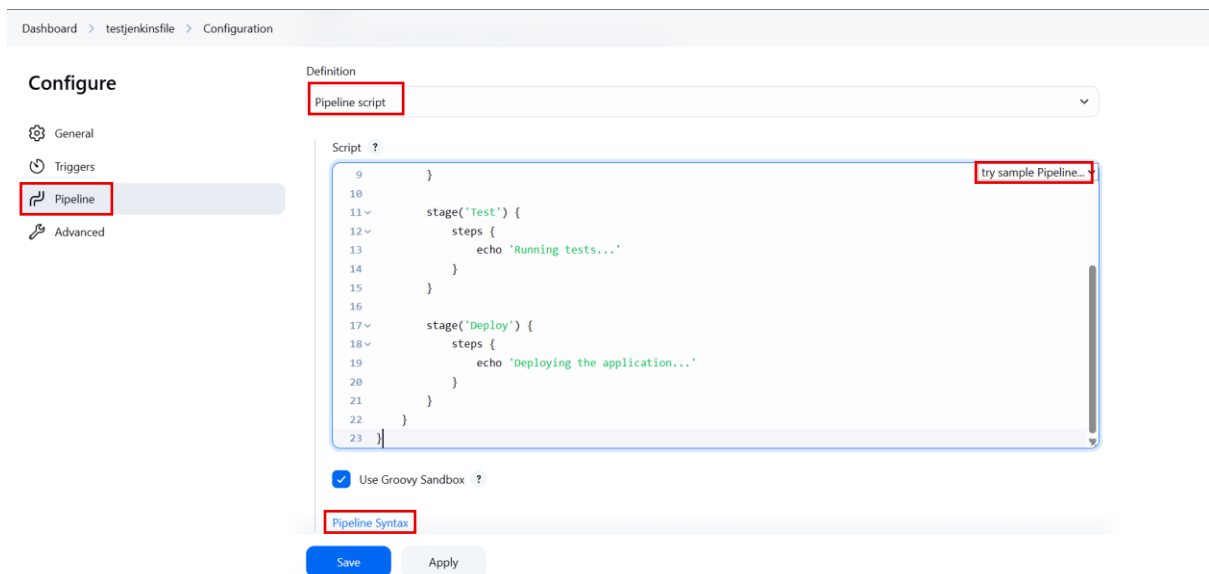
testjenkinsfile

Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a

OK

- Under the pipeline, in definition, select the option Pipeline script.
- Select the try sample pipeline



- Click Save.
- Now click on build now.
- Your #1 has created.
- Here, there is a option called stage to track the pipeline.
- Click on the stage to monitor the pipeline.



- Now we can see the console output.

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Started by user Aman Rathor

[Pipeline] Start of Pipeline

[Pipeline] node

Running on Jenkins in /var/lib/jenkins/workspace/testjenkinsfile

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Build)

[Pipeline] echo

Building the application...

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Test)

[Pipeline] echo

Running tests...

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Deploy)

[Pipeline] echo

Deploying the application...

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

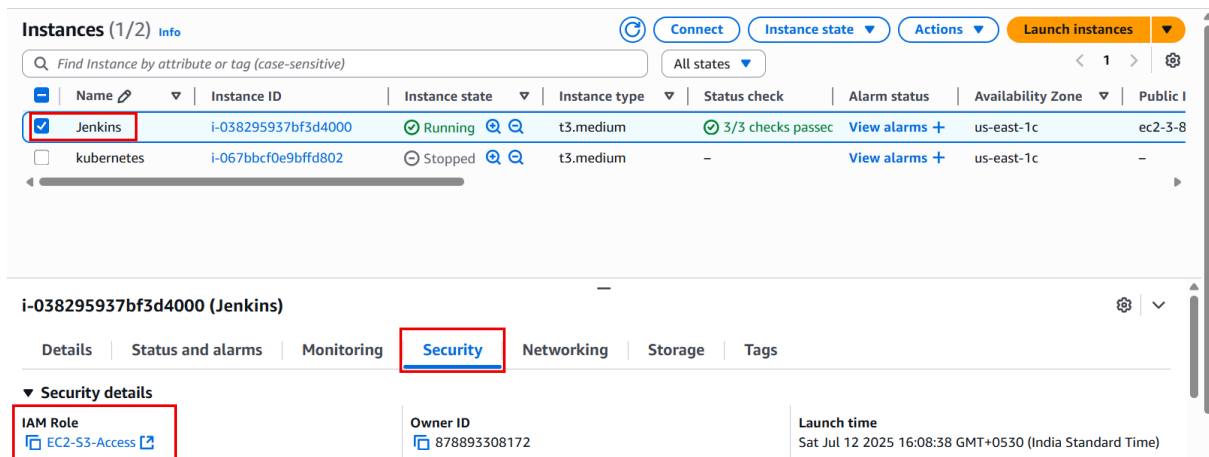
Finished: SUCCESS

Jenkins Permission

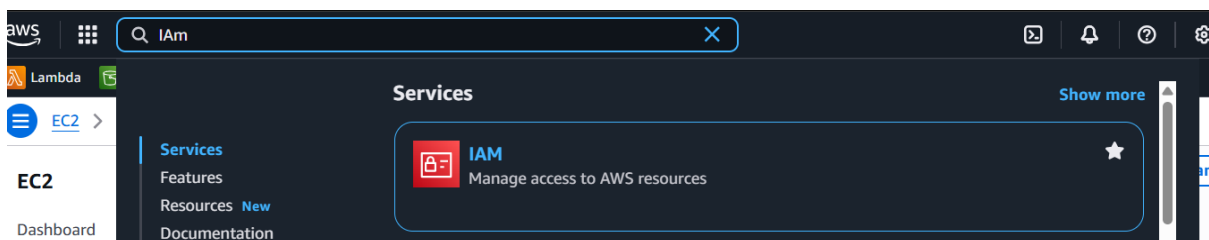
Summary of the lab

This part of the lab configures AWS IAM permissions for an EC2 instance to access S3. From the EC2 console, the instance is checked for an IAM role. If absent, a new IAM role is created from the IAM console, granting either full or read-only S3 access. After naming and creating the role, it is attached to the instance via the **Modify IAM Role** option under instance actions. This setup is crucial when Jenkins pipelines need to access S3 buckets securely using AWS CLI commands from the EC2 instance where Jenkins is hosted.

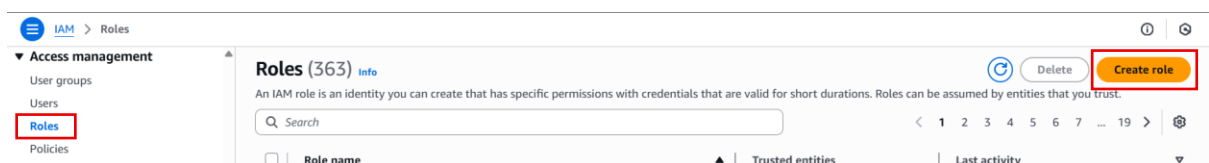
- In this file we will give the permission for S3 buckets from EC2 console.
- Go to the EC2 console.
- Select the instance
- Click on the security.
- Check for IAM Role



- In case if it is not visible.
- Navigate to IAM



- Click on the role.
- Click on the create role.



- Select AWS Services.

- Select EC2.

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Select trusted entity Info

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
EC2

Choose a use case for the specified service.

Use case
☒ **EC2**
Allows EC2 instances to call AWS services on your behalf.

- Click Next.
- Give the permission for S3.
- Here you can give full permission or only read permission.

Add permissions Info

Permissions policies (1/1466) Info

Choose one or more policies to attach to your new role.

Search S3 × **Filter by Type** All types 90 matches

☐	Policy name ?	Type	Description
☒	AmazonS3FullAccess	AWS managed	Provides full access to all buckets via t...
☐	AmazonS3ReadOnlyAccess	AWS managed	Provides read only access to all bucket...
☐	AmazonDMSRedshiftS3Role	AWS managed	Provides access to manage S3 settings ...

- Click Next.
- Give the Role Name.

Role details

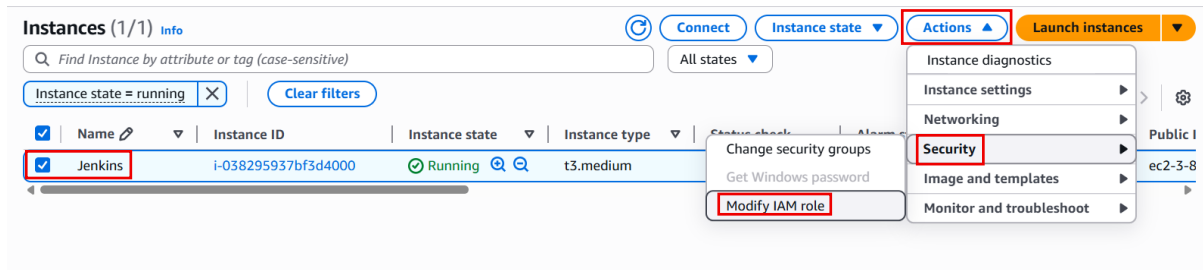
Role name
Enter a meaningful name to identify this role.
EC2-IAM

Maximum 64 characters. Use alphanumeric and '+=, @-/_![]!#\$%^&*()' characters.

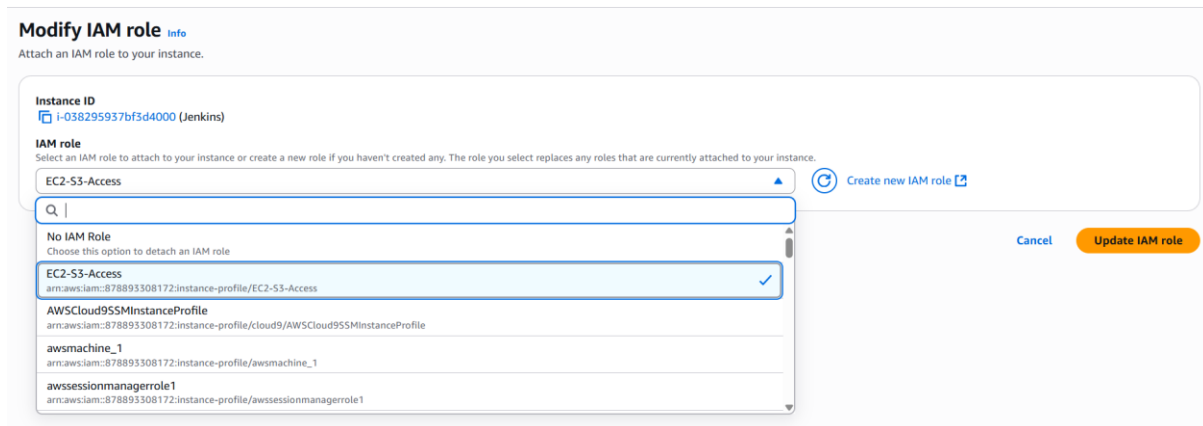
Description
Add a short explanation for this role.
Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=, @-/_![]!#\$%^&*()' characters.

- Click Create Role.
- Go to **EC2 > Instances > Actions > Security > Modify IAM Role**



- Select the name that you have given.
- Click on Update IAM role.



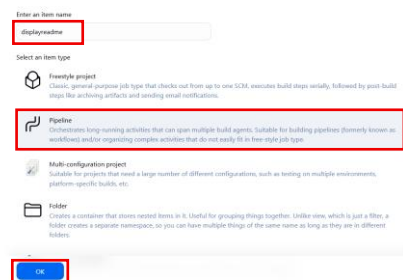
- Your IAM role for EC2 will be accessible.

Writing Jenkinsfile the Easy Way

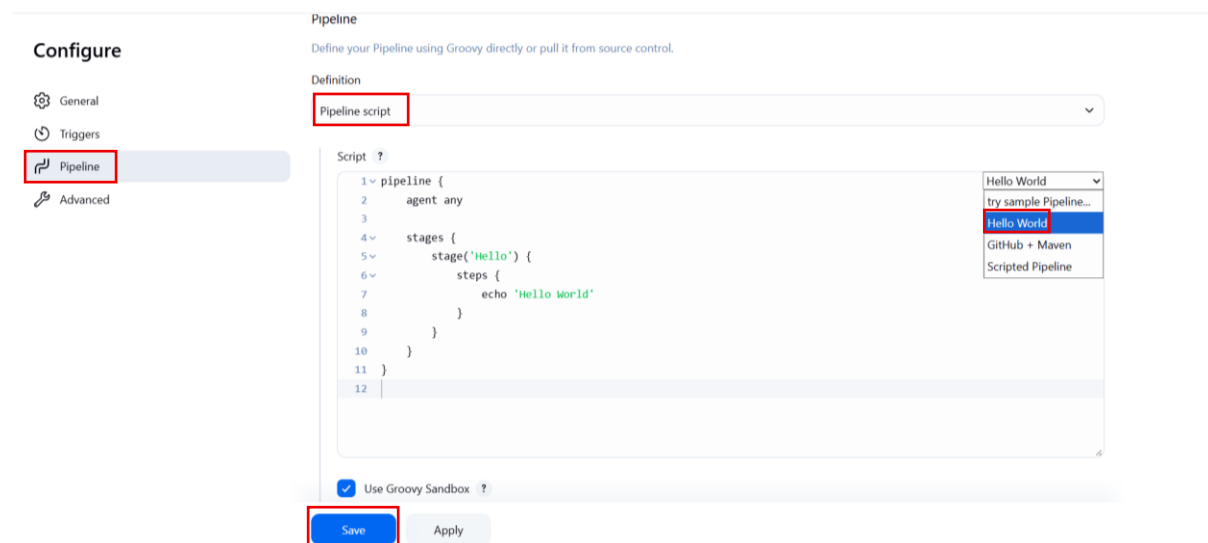
Summary of the file

This lab teaches how to automate Jenkinsfile creation using the **Pipeline Syntax Generator**. A new pipeline job is created, and the user uses Git integration to clone a GitHub repository (e.g., kubernetescode). Additional stages are added to read the README file and list S3 buckets using the AWS CLI. The Jenkinsfile is written in declarative syntax, making it easy to follow. The stages demonstrate source code checkout, file content display, and external resource listing, offering a practical Jenkins pipeline example that incorporates Git and AWS commands in a simple, modular format.

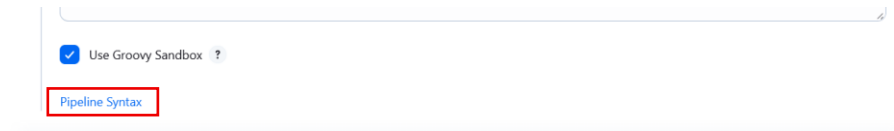
- Here we will [Kubernetescode Repository](#).
- Here we will display the README file.
- Create a new Item.
- Give the name.
- Select the pipeline option.
- Click OK



- Under the pipeline, in definition, select the option Pipeline script.
- Select the Hello World



- Now click on the pipeline syntax
- This pipeline syntax is used to generate the code.



- A new tab will open.
- Now in sample step, select git: Git

Steps

Sample Step

checkout: Check out from version control

- Now enter the details

checkout ?

SCM

Git

Repositories ?

Repository URL ?

https://github.com/riteshbehal/kubernetescode.git

Credentials ?

- none -

+ Add

Advanced

- In the Branch Specifier option, choose your branch main or master.

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add

☒ Include in polling? ?

☒ Include in changelog? ?

Generate Pipeline Script

- Click generate pipeline script
- It will generate the code.

```
checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs: [[url: 'https://github.com/riteshbehal/kubernetescode.git']])
```



- Copy the code and paste in the pipeline.
- Now go back to the job.
- Edit the Script.
- Now in sample step, select sh: Shell Script
- Copy this and paste it.

Steps

Sample Step

sh: Shell Script

sh

Shell Script ?

```
cat README.md
```

Advanced ▾

Generate Pipeline Script

```
sh 'cat README.md'
```

- This is a pipeline script.
- Now we will create a new stage to display the S3 bucket.
- Edit the script.
- Given is the pipeline.

```
pipeline {
```

```
    agent any
```

```
    stages {
```

```
        stage('Cloning Repo') {
```

```
            steps {
```

```
                checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs:
[[url: 'https://github.com/riteshbehal/kubernetescode.git']])
```

```
            }
```

```

    }

    stage('Display README File') {

        steps {

            sh 'cat README.md'

        }

    }

    stage('List S3 Buckets') {

        steps {

            sh 'aws s3 ls'

        }

    }

}

```

- Click Save.
- Now click on build now.
- Your #1 has created.
- Here, there is a option called stage to track the pipeline.
- Click on the stage to monitor the pipeline.

Dashboard > displayreadme >

Status

</> Changes

▷ Build Now

⚙️ Configure

🗑️ Delete Pipeline

📁 Stages

✎️ Rename

🔗 Pipeline Syntax

displayreadme

Add description

Permalinks

Builds

Today

#1 4:45 PM

Dashboard > displayreadme > Stages

Stages

July 12, 2025

✓ #1
 22:15 - 4.5s

List S3 Buckets
 2.1s

○ - ✓ - ✓ - ✓ - ○

- Now we can see the console output.

Status

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

 Console Output Download Copy

View as plain text

```
Started by user Aman Rathor
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/displayreadme
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Cloning Repo)
[Pipeline] checkout
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/riteshbehal/kubernetescode.git
> git init /var/lib/jenkins/workspace/displayreadme # timeout=10
Fetching upstream changes from https://github.com/riteshbehal/kubernetescode.git
> git --version # timeout=10
> git --version # 'git version 2.47.1'
> git fetch --tags --force --progress -- https://github.com/riteshbehal/kubernetescode.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git config remote.origin.url https://github.com/riteshbehal/kubernetescode.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
```

```
2025-05-14 05:26:48 source-buckets-22
2025-05-09 09:25:29 source-cicd-bucket1
2025-06-20 15:14:05 sourcedemo20062025
2025-05-31 11:55:55 space-finder-frontend-0affe087abbf
2025-06-02 12:11:22 space-finder-photos-0affe087abbf
2024-12-09 13:35:19 stack-general
2025-05-17 06:11:31 staticwebcodebucket
2025-01-29 10:47:01 testglue9876
2025-01-29 11:13:26 testgluetarget9876
2025-06-11 06:32:40 testpss3
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins file Post Section Demo

Summary of the Lab

This part demonstrates how Jenkins handles failed stages using the post section. The existing pipeline job is modified by intentionally making the "List S3 Buckets" stage fail (removing `ls`). A `post { failure { ... } }` block is added to log a custom message ("Job Failed") when a failure occurs. After building the job, the console output confirms the error and displays the post-failure message. This feature is useful for notifications or cleanup actions regardless of pipeline success. Re-adding `ls` restores success, showing how post blocks enhance pipeline resilience and monitoring.

- Here we will see the even if our stage fails but we can build our job successfully using post stage.
- We will continue with the previous job that we have create.
- Go to the job(displayreadme).
- Click config.
- Now in the script, we will create a post stage.
- So, the change that I have made are:
- in the List S3 bucket stage.
- Remove the `'ls'` to make the stage fail.
- Add the post stage.
- Below is the script given:

```
pipeline {
    agent any

    stages {
        stage('Cloning Repo') {
            steps {
                checkout scmGit(branches: [[name: '*/main']], extensions: [], userRemoteConfigs:
[[url: 'https://github.com/riteshbehal/kubernetescode.git']])
            }
        }
        stage('Display README File') {
            steps {
                sh 'cat README.md'
            }
        }
    }
}
```

```

    }

    stage('List S3 Buckets') {

        steps {

            sh 'aws s3'

        }

    }

}

post {

    failure {

        echo "Job Failed"

    }

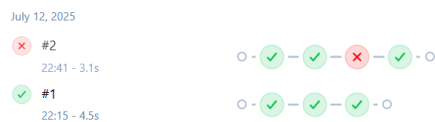
}

}

```

- Click on build now.
- Click on stage
- Click on #1 to see the Console Output.

Stages



- Console Output

```

## ArgoCD installation

Install ArgoCD in your Kubernetes cluster following this link - https://argo-cd.readthedocs.io/en/stable/getting\_started/

[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (List S3 Buckets)
[Pipeline] sh
+ aws s3

usage: aws [options] s3 <subcommand> [parameters]
aws: error: too few arguments
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Job Failed
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 252
Finished: FAILURE

```

- Go to config
- Make the changes
- Now add 'ls' in this list s3 stage.

```

14     }
15     stage('List S3 Buckets') {
16         steps {
17             sh 'aws s3 ls'
18         }

```

- and Click on Build Now.

Dashboard > displayreadme > #3 > Pipeline Overview

✓ < #3 Rerun ...

Manually run by Aman Rathor Started 12 sec ago Queued 1 ms Took 2.2 sec

Graph

Search

- ✓ Cloning Repo 0.22s
- ✓ Display README File 0.33s
- ✓ List S3 Buckets 1.1s

✓ List S3 Buckets 1.1s Started 39s ago Jenkins

aws s3 ls 1.0s

```

0 + aws s3 ls
1 2025-06-25 03:40:12 agent-bucket12
2 2025-05-22 07:12:01 aws-cloudtrail-logs-878893308172-17e4408d
3 2025-05-22 07:09:09 aws-cloudtrail-logs-878893308172-4df355fa

```

Dashboard > displayreadme > #3

Status

- </> Changes
- Console Output
- Edit Build Information
- Delete build '#3'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- More...

✓ Console Output Download Copy View as plain text

Started by user Aman Rathor

```

[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/displayreadme
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Cloning Repo)
[Pipeline] checkout
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/displayreadme/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/riteshbehal/kubernetescode.git # timeout=10
Fetching upstream changes from https://github.com/riteshbehal/kubernetescode.git
> git --version # timeout=10
> git --version # 'git version 2.47.1'

```

```

2024-12-09 13:35:19 stack-general
2025-05-17 06:11:31 staticwebcodebucket
2025-01-29 10:47:01 testglue9876
2025-01-29 11:13:26 testgluetarget9876
2025-06-11 06:32:40 testpss3
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```